Аккредитованное образовательное частное учреждение
высшего образования
«Московский финансово-юридический университет МФЮА»
(МФЮА)

КАФЕДРА «ИНФОРМАЦИОННЫЕ СИСТЕМЫ И ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

по дисциплине: «Разработка программных модулей»

на тему: «Разработка модуля управления каталогом товаров для интернет-магазина»

Специальность: 09.02.07
Информационные системы и программирование
Профиль: Программист

Автор работы: Чемоданов Никита Сергеевич
Курс: 4 Группа: 23ИПо8381 (ИПо-41)
Руководитель работы: Колпащиков Кирилл Александрович
Подписи и даты: _____

Киров — 2026

СОДЕРЖАНИЕ

Введение.....3
1 Теоретические основы и анализ управления товарным каталогом.....5
1.1 Особенности ведения каталога товаров интернет-магазина.....5
1.2 Подходы к автоматизации и требования к модулю.....6
1.3 Обоснование выбора технологий.....8
2 Проектирование модуля управления каталогом.....11
2.1 Пользовательские сценарии и функциональная модель.....11
2.2 Проектирование базы данных и ограничений целостности.....13
2.3 Архитектура приложения и проектирование интерфейса.....16
3 Разработка и тестирование модуля.....18
3.1 Реализация операций с товарами и категориями.....18
3.2 Реализация поиска, фильтрации и веб-интерфейса.....19
3.3 Тестирование, анализ результатов и ограничения решения.....21
Заключение.....24
Список использованных источников.....26
Приложение А. Экраны и результаты демонстрации.....29

ВВЕДЕНИЕ

Управление ассортиментом предполагает согласованное хранение сведений о товарах, их принадлежности к категориям, цене и доступном количестве. Для менеджера недостаточно иметь перечень названий: необходимо находить нужную запись, отличать сходные позиции, изменять значения и получать понятное сообщение при ошибке. Если правила ввода и хранения различаются между отдельными операциями, каталог может содержать повторяющиеся артикулы, недопустимые количества или ссылки на отсутствующие категории. Поэтому актуальность учебного проекта связана с реализацией проверяемого модуля, в котором пользовательские действия согласованы с ограничениями данных.

Работа выполняется по дисциплине «Разработка программных модулей» для специальности 09.02.07 «Информационные системы и программирование». Выданное задание определяет операции добавления, просмотра, изменения и удаления товаров, поиск, фильтрацию, сортировку и хранение данных. В качестве документационной основы в задании назван ГОСТ 19.201–78 [1]. Структура и базовое оформление записки приняты по предоставленным методическим указаниям МФЮА [2]. Объём проекта ограничен управлением каталогом: заказы, расчёты с покупателями и доставка не рассматриваются.

Тема связана с несколькими направлениями учебной и технической литературы. А. Дауни показывает организацию данных и поведения с помощью типов Python [3]. Х. Персиваль и Б. Грегори рассматривают отделение прикладных операций от кода взаимодействия и границы транзакций [5]. Г. Уилсон объясняет проектирование через небольшие воспроизводимые примеры [7]. В работах по тестированию сопоставляются проверки отдельных функций и пользовательских сценариев, а также роль тестов при изменении программы [6; 8]. Эти подходы применяются с учётом масштаба учебного модуля: сложная архитектура сама по себе не является целью.

Цель работы — разработать и проверить модуль управления каталогом товаров интернет-магазина, обеспечивающий ведение товаров и категорий, поиск и совместное применение условий отбора. Для достижения цели поставлены задачи: проанализировать предметную область и требования; обосновать технологии; спроектировать сущности, ограничения и интерфейс; реализовать операции и обработку ошибок; подготовить демонстрационные данные; проверить положительные и отрицательные сценарии; оценить результаты и ограничения решения.

Объект исследования — процесс ведения товарного каталога интернет-магазина. Предмет исследования — структуры данных, алгоритмы обработки запросов и программные средства автоматизации хранения, изменения и поиска сведений о товарах и категориях. В качестве учебного ассортимента используются канцелярские товары. Сведения не относятся к обследованному предприятию; цены и остатки в демонстрационном наборе условные.

В работе применены анализ документации и учебной литературы, сравнение вариантов решения, моделирование сущностей и пользовательских сценариев, программирование и функциональное тестирование. Ожидаемый результат оценивается по возможности выполнить согласованные операции и сохранить установленные свойства данных. Измерение экономического эффекта и доказательство пригодности к промышленной нагрузке в задачу не включены. Такой способ оценки позволяет связать выводы с наблюдаемым поведением программы.

Практическая значимость работы состоит в создании запускаемого локального приложения с исходным кодом, инструкцией и воспроизводимыми проверками. Модуль предоставляет веб-страницы и JSON API, использующие общие операции каталога. Записка включает три главы. В первой рассмотрены предметная область, требования и выбор технологий; во второй — модель данных, сценарии и архитектура; в третьей — реализация, демонстрационный набор и фактические результаты тестирования. Приложение содержит реальные экраны программы.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ УПРАВЛЕНИЯ ТОВАРНЫМ КАТАЛОГОМ

1.1 ОСОБЕННОСТИ ВЕДЕНИЯ КАТАЛОГА ТОВАРОВ ИНТЕРНЕТ-МАГАЗИНА

Товарный каталог в рамках проекта представляет собой совокупность записей об ассортименте. Каждая запись содержит название, артикул, категорию, цену, описание и текущее количество. Название служит для распознавания и поиска позиции, но не обязано быть уникальным. Артикул позволяет однозначно отличить товар от других позиций. Например, два варианта ручки могут иметь близкие названия, однако различаться цветом, толщиной линии и артикулом. Поэтому запрещать все совпадения названий было бы излишним, а разрешать повторный артикул — противоречить принятому правилу идентификации.

Необходимо различать идентификатор записи и артикул. Идентификатор создаётся базой и используется в связях и адресах карточек. Артикул задаётся менеджером и включается в пользовательское представление. Изменение артикула не должно создавать новый объект вместо существующего: при редактировании идентификатор сохраняется. Разделение технического ключа и пользовательского обозначения делает сценарий изменения однозначным и позволяет проверять уникальность независимо от порядка записей.

Категория объединяет товары по назначению. Её название и описание хранятся отдельно, а товар содержит ссылку на соответствующую запись. Если категория переименована, товары продолжают относиться к ней по прежнему идентификатору. Название категории не требуется вручную исправлять у каждой позиции. Для первой версии выбрана одна категория на товар. Это упрощает форму и условия отбора; иерархическая классификация потребовала бы дополнительных правил перемещения и удаления узлов.

Цена и количество отражают разные свойства. Нулевая цена допускается, отрицательная отклоняется. Остаток определяется как целое число единиц, не меньшее нуля. Товар с нулевым остатком сохраняется в общем каталоге, но не входит в выдачу при фильтре наличия. В отличие от текущего остатка журнал складских операций должен описывать причины изменения количества: приход, списание, резервирование. В принятой модели такой журнал отсутствует, поэтому изменение числа в карточке не является полноценным складским учётом.

Представление предметных сущностей в программе связано с выбором типов и операций. В учебнике Think Python классы рассматриваются как способ объединить пользовательский тип и связанное с ним поведение, а словари — как отображение ключей в значения [3, гл. 10, 14]. В проекте сущности представлены моделями, а допустимые данные запроса — отдельными схемами. Это позволяет различать хранимую запись, введённые значения и данные, подготовленные для ответа, даже если часть их полей совпадает.

Основной пользователь модуля — менеджер каталога. Его работа начинается с создания категории и товара, продолжается поиском и корректировкой и может завершаться удалением записи. Для этих действий важна обратная связь: сообщение о сохранении, объяснение конфликта и возможность исправить ввод. Следовательно, качество модуля определяется не только наличием CRUD-операций, но и тем, сохраняет ли интерфейс смысл действий при ошибках и повторных запросах.

1.2 ПОДХОДЫ К АВТОМАТИЗАЦИИ И ТРЕБОВАНИЯ К МОДУЛЮ

Для учебной задачи рассмотрены три подхода: ведение перечня в таблице, настройка готовой системы магазина и разработка самостоятельного модуля. Сравнение выполнено по требованиям проекта. Испытание конкретных коммерческих продуктов, расчёт стоимости внедрения и измерение времени работы менеджера не проводились. Поэтому выводы характеризуют

соответствие выбранной задаче, а не универсальное преимущество одной технологии.

Табличный перечень позволяет описать строки товаров и столбцы свойств. Однако связанные категории, уникальность артикула и поведение при ошибке необходимо организовать отдельными правилами. Таблица полезна как исходное представление данных, но не показывает реализацию обработки HTTP-запросов и серверной проверки. Готовая система требует выбора продукта, настройки и, при необходимости, расширения. Этот путь решает задачу внедрения системы, тогда как цель курсовой — спроектировать и объяснить собственный программный модуль на Python.

Отдельный веб-модуль даёт возможность явно связать каждую операцию со схемой данных, обработчиком и проверкой. Ответственность за корректность и удобство при этом лежит на разработчике. В данной работе выбран именно этот вариант: он соответствует заданию и позволяет проследить поведение программы по исходному коду. Проектное сравнение подходов представлено в таблице 1. Принцип изучения небольших работающих примеров, рассмотренный Г. Уилсоном, поддерживает такой учебный масштаб решения [7].

Таблица 1 — Проектное сравнение подходов

Подход	**Что организуется**	**Соответствие задаче**
Таблица товаров	Столбцы свойств, правила ввода и связи нужно настроить	Подходит для исходного перечня; HTTP-модуль не реализует
Готовая система	Выбор продукта, настройка или расширение	Возможный прикладной путь; самостоятельная разработка не показана
Отдельный модуль	Собственные схема, операции, формы и проверки	Выбран для изучения Python и FastAPI

Функциональные требования включают создание, просмотр, изменение и удаление товаров и категорий, подробную карточку товара, поиск по названию, фильтрацию категории и цены, признак наличия, сортировку и постраничный вывод. Условия отбора должны сочетаться. Поиск не зависит от регистра русского текста; ценовые границы включают равные значения. Страница списка содержит не более десяти товаров, а ссылки переходов сохраняют выбранные условия.

Требования к корректности задают ограничения: товар связан с существующей категорией; артикул и нормализованное имя категории уникальны; цена и остаток неотрицательны; остаток целочисленный. Удаление занятой категории отклоняется. После остановки и запуска обычного сервера данные сохраняются. Ошибка запроса не должна приводить к частичному изменению другой записи. Эти требования проверяются наблюдаемым результатом и состоянием базы, а не только наличием элементов формы.

При проверке важно отличать ожидание от факта. В книге The Fuzzing Book вводное рассмотрение тестирования связывает проверку с явным ожидаемым поведением и подчёркивает ограниченность отдельных примеров [10]. Для проекта это означает необходимость отрицательных случаев наряду с успешным сохранением. Например, недостаточно создать товар: требуется проверить повторный артикул, неправильную цену, отсутствующую категорию и сохранение прежних данных после отказа.

1.3 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

Основной язык разработки — Python 3.12. Он позволяет описать входные данные, модели и операции в одном проекте. Используемые пакеты закреплены в файлах зависимостей, а виртуальное окружение отделяет их от системных библиотек. Для воспроизведения результатов важны не только исходники, но и сведения об окружении и последовательности запуска; соответствующий

подход к происхождению результатов рассматривается в Research Software Engineering with Python [4].

FastAPI выбран для обработки HTTP-запросов и предоставления JSON API. Пользовательские страницы формируются сервером с Jinja2. Документация FastAPI описывает передачу объекта запроса, имени шаблона и контекста в TemplateResponse, а также подключение статических файлов [13]. Такая организация подходит для небольшого интерфейса менеджера: после отправки формы сервер проверяет данные и возвращает страницу либо перенаправление. Основные операции не зависят от JavaScript.

Для работы с данными применяется SQLAlchemy, для входных схем — Pydantic. Сессия SQLAlchemy связывает загруженные объекты с транзакцией и предоставляет явные commit и rollback [14]. Строгий режим Pydantic ограничивает неявные преобразования типов [16]. В API количество и идентификатор категории принимаются целыми JSON-числами. HTML передаёт строки, поэтому адаптер формы отдельно разбирает допустимую целочисленную запись, затем применяет ту же схему.

СУБД выбрана с учётом локальной демонстрации. Согласно документации Python, SQLite хранит данные в файле и не требует отдельного серверного процесса [11]. В задании среди источников названа PostgreSQL. Её документация использована для анализа таблиц, ключей и ограничений [12]. При этом цель задания не закрепляет конкретную СУБД, а фактически реализованная версия использует SQLite. Это решение уменьшает число действий при установке на Mac и позволяет создавать отдельные файлы для тестов.

Выбор SQLite не означает доказанной пригодности к произвольной нагрузке. Перенос на PostgreSQL не выполнялся. Несмотря на общий ORM, SQLite-специфичные проверки typeof и GLOB, параметры подключения и PRAGMA потребовали бы пересмотра. Следовательно, возможность описать сущности через SQLAlchemy не равна автоматической переносимости всей

схемы. Условия многопользовательского использования должны оцениваться отдельной проверкой.

Для автоматических проверок используются pytest и тестовый HTTP-клиент; реальные браузерные действия выполняются через Playwright [17; 19]. Сервер запускается Uvicorn с привязкой к локальному адресу и заданному порту [20]. Вместе эти инструменты покрывают согласованные задачи разработки, демонстрации и функциональной проверки. Их выбор не добавляет в проект механизмов, не входящих в требования, и сохраняет возможность объяснить путь отдельного запроса.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ УПРАВЛЕНИЯ КАТАЛОГОМ

2.1 ПОЛЬЗОВАТЕЛЬСКИЕ СЦЕНАРИИ И ФУНКЦИОНАЛЬНАЯ МОДЕЛЬ

Пользовательские сценарии начинаются с формирования структуры каталога. Менеджер создаёт категорию, вводя название и необязательное описание. Если эквивалентное нормализованное название уже существует, сервер отклоняет запись и возвращает форму. После успешного сохранения категория доступна для выбора при создании товара. При пустом справочнике форма товара предлагает сначала создать категорию, поскольку ссылка на неё обязательна.

Создание товара включает название, артикул, категорию, цену и остаток. Описание допускается не заполнять. После проверки сервер сохраняет запись и открывает карточку с результатом. При редактировании форма содержит текущие значения; сохраняется тот же идентификатор. Если новый артикул занят, изменённый товар не должен частично получить другую цену или название. Соответствующие проверки выполняются до изменения объекта и повторяются ограничением уникальности при записи.

Поиск сочетается с категорией, нижней и верхней границами цены, наличием и сортировкой. Отключённое условие не ограничивает выдачу. Признак наличия означает количество больше нуля, а не положительную цену. При переходе на следующую страницу сохраняются все условия. Пустая выдача отличается от отсутствия данных в новом каталоге: в первом случае предлагается изменить отбор, во втором — создать первые записи. Связь требований и признаков корректности приведена в таблице 2.

Таблица 2 — Связь требований с проверками

Требование	**Реализация**	**Признак корректности**
CRUD товара	Форма, карточка, API	Изменения видны после чтения; ID сохраняется
Категории	Справочник и связь	Повторное имя и удаление занятой категории отклоняются
Поиск	Нормализованное имя	Русский регистр не меняет результат
Фильтры	Условия общего запроса	Категория, цена и наличие применяются вместе
Пагинация	LIMIT 10 и OFFSET	120 страниц для 1200 товаров
Неправильный ввод	Схемы и форма	Статус 422; заполненные значения сохранены
Данные после старта	Файловая SQLite	Повторное открытие не удаляет записи

Удаление товара разделено на открытие подтверждения и изменение базы. GET показывает сведения и выбор действия; только POST выполняет удаление. Отмена возвращает карточку. Удаление занятой категории вместо подтверждения показывает причину запрета и ссылку на её товары. Сервер проверяет это правило повторно при POST, поскольку состав категории мог измениться после открытия страницы.

Для маршрутов определён согласованный результат. Неправильные данные дают статус 422, отсутствие объекта — 404, конфликт уникальности или связи — 409. В API возвращается JSON, в интерфейсе — HTML с понятным сообщением. Успешное создание через API возвращает 201, удаление — 204 без тела. После успешной HTML-формы используется 303 и следующий GET. Различие представлений сохраняет единые правила, но учитывает способ обращения пользователя.

Сценарии связываются с проверками ещё на этапе проектирования. В работе Х. Персиваля функциональный тест описывает поведение с точки зрения пользователя, а более узкие проверки помогают локализовать отказ [6, гл. 4]. В проекте применены оба масштаба: запросы к API и HTML проверяют наблюдаемый результат, проверки базы — ограничения хранения, а браузерный прогон — выполнение последовательности действий на страницах.

2.2 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ И ОГРАНИЧЕНИЙ ЦЕЛОСТНОСТИ

Модель включает сущности «Категория» и «Товар» со связью один ко многим. Категория имеет первичный ключ id, название, нормализованное название и описание. Товар содержит id, название, его нормализованную копию, артикул, описание, category_id, цену в копейках, количество и даты создания и изменения. Каждая товарная запись ссылается на одну существующую категорию. Логическая структура показана на рисунке 1.

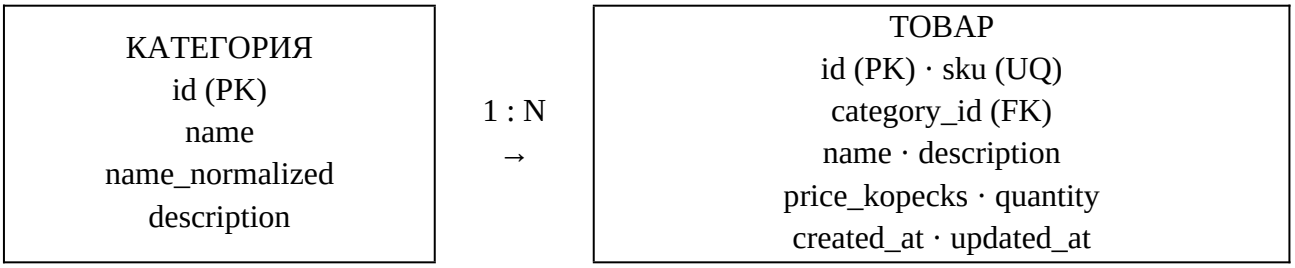


Рисунок 1 — Логическая структура данных каталога

Источник: составлено по модели и исходному коду проекта.

Разделение справочника и товаров предотвращает повторение пользовательского названия категории в каждой товарной строке. Это делает переименование одной операцией над справочником. Принципы первичного и внешнего ключей и ограничения уникальности рассмотрены в документации PostgreSQL [12]. В проекте они реализованы средствами SQLite и SQLAlchemy. Для внешнего ключа задан RESTRICT: наличие связанных товаров блокирует удаление категории.

Цена хранится целым числом копеек. Строка рублей проверяется по допустимому формату и преобразуется через Decimal. Принимаются точка и

запятая, не более двух знаков после разделителя; лишняя точность отклоняется без скрытого округления. Документация decimal объясняет возможность точного представления десятичных значений в отличие от ряда значений float [11]. При выводе целые копейки снова преобразуются в строку рублей с двумя дробными знаками.

Количество ограничено неотрицательным целым в диапазоне SQLite. Входные схемы отвергают логические и дробные значения. На уровне хранения CHECK проверяет целочисленный вид записанного значения и неотрицательность. Уникальность артикула и нормализованного имени категории обеспечивается индексами. Предварительная проверка приложения улучшает сообщение пользователю, но не заменяет ограничение базы при конкурирующих запросах.

Для поиска используется дополнительное нормализованное имя: NFKC, удаление крайних пробелов и casefold. Согласно документации unicodedata, NFKC сочетает совместимостное разложение с канонической композицией [11]. Исходное название сохраняется для отображения. Служебные поля заполняются моделями при записи через приложение. Это осознанное дублирование производного значения; его не следует смешивать с нормализацией реляционной схемы. Основные поля и правила сведены в таблицу 3.

Таблица 3 — Основные поля модели данных

Сущность и поле	**Представление**	**Правило**
Category.id	Целое	Первичный ключ
Category.name	Строка до 100 символов	Непустое; уникальна нормализованная форма
Category.description	Текст или NULL	Не более 2000 символов
Product.id	Целое	Первичный ключ
Product.name	Строка до 200 символов	Непустое; нормализованная копия для поиска
Product.sku	Строка до 64 символов	Уникальный; латинские буквы, цифры, дефис, подчёркивание
Product.category_id	Целое	Внешний ключ; категория существует; RESTRICT
Product.price_kopecks	Целые копейки	От 0 до 2^63 − 1; при вводе рубли строкой
Product.quantity	Целое количество	От 0 до 2^63 − 1
Product.description	Текст или NULL	Не более 2000 символов
Product.created_at, updated_at	Дата и время	Хранятся в UTC; обновление при изменении

Связь таблиц: categories.id → products.category_id, один ко многим. Удаление занятой категории запрещено. Нормализованные поля служебные и не входят в публичную схему API.

Описание после удаления крайних пробелов может стать пустым и сохраняется как NULL. Даты фиксируются в UTC; SQLite хранит их без встроенной информации о часовом поясе, поэтому это соглашение соблюдает приложение. API добавляет обозначение UTC, а браузер может отображать местное время. Остаток хранит текущее число единиц без истории движений.

Эти договорённости определяют смысл данных и должны сохраняться при развитии системы.

2.3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ И ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСА

Компоненты организованы по назначению: database.py создаёт движок и сессии; models.py описывает хранимые сущности; schemas.py проверяет ввод; catalog.py реализует общие операции; маршруты API и страниц адаптируют запросы и ответы. Шаблоны и статические файлы отвечают за представление. Ответственность компонентов приведена в таблице 4, путь запроса — на рисунке 2.

Таблица 4 — Ответственность компонентов

Компонент	**Назначение**
database.py	Движок, внешний ключ, сессия на запрос
models.py / schemas.py	Таблицы, ограничения и проверка входных данных
catalog.py	Общие операции, транзакции, поиск и фильтры
routers/api.py	JSON-контракт и статусы HTTP
routers/pages.py	HTML-формы, сообщения и перенаправления
templates / static	Представление, оформление и местное время
demo_data.py / seed_demo.py	Воспроизводимые примеры, backup и загрузка

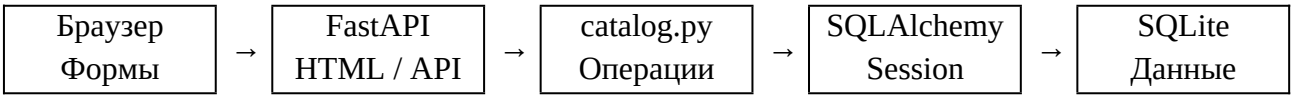


Рисунок 2 — Обработка запроса в приложении

Источник: составлено по модели и исходному коду проекта.

Такое разделение соответствует идее отличать прикладную логику от кода взаимодействия, рассмотренной в Architecture Patterns with Python [5, гл. 4]. При этом проект не копирует всю архитектуру книги. Отдельные репозитории, очередь событий и собственная абстракция Unit of Work не

введены: для текущего объёма общие функции и сессия SQLAlchemy достаточны. Упрощение не отменяет необходимости определить место проверки и границу транзакции.

HTML и API вызывают одинаковые функции каталога. Например, проверка занятости категории находится в общей операции, поэтому правило не зависит от того, каким интерфейсом воспользовался менеджер. Для каждого запроса выдаётся собственная сессия, успешная запись подтверждается явно, конфликт целостности сопровождается rollback. Идея атомарной операции обсуждается в книге Персиваля и Грегори [5, гл. 6], а конкретное поведение сессии определяется SQLAlchemy [14].

Интерфейс включает список товаров с фильтрами, карточку, общую форму создания и изменения, подтверждение удаления и страницы категорий. На широком экране меню расположено слева. На узком — сверху, поля формы выстраиваются последовательно. Таблица прокручивается в собственном контейнере. Для полей предусмотрены подписи, для ошибок — заметное сообщение с сохранённым вводом. Основные действия доступны без JavaScript; скрипт дополняет отображение дат местным временем.

Доступ к базе отделён от сохранения состояния в файлах проекта. В книге Г. Уилсона различаются восстановление объекта из сохранённого представления и работа с базой, позволяющей выбирать записи по ключам [7]. Здесь рабочие данные находятся в SQLite, а код и воспроизводимый генератор примеров — в Git. Это позволяет обновлять приложение, не подменяя пользовательский каталог, и создавать отдельные базы для проверок.

3 РАЗРАБОТКА И ТЕСТИРОВАНИЕ МОДУЛЯ

3.1 РЕАЛИЗАЦИЯ ОПЕРАЦИЙ С ТОВАРАМИ И КАТЕГОРИЯМИ

Точкой входа служит app/main.py. При запуске lifespan создаёт отсутствующие таблицы. Путь к data/catalog.db вычисляется от папки проекта. Повторная инициализация сохраняет записи, но не выполняет миграцию существующей схемы. Локальный запуск выполняется Uvicorn с адресом 127.0.0.1 и портом 8000; назначение параметров соответствует документации сервера [20]. Локальная работа приложения на macOS дополнительно проверена студентом.

API содержит десять операций: получение списка, создание, чтение одной записи, изменение и удаление для каждой сущности. Создание товара сначала проверяет схему, наличие категории и уникальность артикула. После добавления объекта выполняется commit, затем обновление его состояния для ответа. Артикул приводится к верхнему регистру. При изменении проверяется существующая запись, новая категория и артикул, после чего заменяются редактируемые поля.

Проверка до присваивания важна для отказов редактирования. Например, попытка заменить артикул занятой строкой не должна одновременно изменить цену. Если конфликт всё же возникает при записи, обработчик IntegrityError откатывает транзакцию и возвращает 409. Документация SQLAlchemy требует учитывать состояние сессии после неуспешной записи [14]. В тестах после конфликтного запроса выполняется корректный, чтобы проверить сохранность данных и возможность продолжить работу.

Удаление категории проверяет наличие связанных товаров. Даже после предварительной проверки внешний ключ остаётся последним ограничением хранения. Удаление товара не должно удалять его категорию. Время изменения обновляется при изменении записи. Эти правила выполняются общими функциями, а маршруты отвечают за преобразование результата в JSON или

HTML. PUT требует полный набор обязательных редактируемых полей; частичное обновление не реализовано.

HTML-форма возвращает введённые значения при 422 и конфликте 409. После успеха браузер получает перенаправление 303 на карточку либо список. RFC 9110 определяет назначение 303 See Other как переход к ресурсу, представляющему результат исходного запроса [18, п. 15.4.4]. Следующий просмотр выполняется GET, поэтому обновление карточки не повторяет сохранение. Открытие страницы подтверждения также не удаляет объект.

Обычные формы передают строки, тогда как схема количества и категории ожидает строгие целые числа. Адаптер допускает только десятичную целочисленную запись подходящей длины, затем передаёт число Pydantic [16]. Строгость JSON-контракта при этом сохраняется. Проверка в браузере помогает пользователю, но сервер повторяет её, поскольку HTTP-запрос может поступить без формы.

3.2 РЕАЛИЗАЦИЯ ПОИСКА, ФИЛЬТРАЦИИ И ВЕБ-ИНТЕРФЕЙСА

Отбор выполняет общая функция list_products. Поиск сравнивает нормализованную строку с нормализованным названием товара. Символы процента и подчёркивания экранируются как буквальный ввод для LIKE. К запросу добавляются выбранные ограничения категории, нижней и верхней цены и количества больше нуля. До выборки проверяется корректность диапазона; минимальная цена не может превышать максимальную.

Сортировка выбирается из фиксированного перечня: название по возрастанию и убыванию, цена по возрастанию и убыванию, недавнее изменение. При равных значениях используется идентификатор, чтобы порядок оставался однозначным. Документация PostgreSQL отдельно указывает, что порядок строк без явно заданной сортировки не гарантируется [12]. В проекте порядок задаётся для каждого разрешённого варианта и проверяется на равных значениях.

Пагинация использует два результата: общее количество подходящих записей и ограниченную страницу данных. Размер страницы равен десяти, смещение определяется номером страницы. Форма фильтрации использует GET, поэтому параметры остаются в адресе. Ссылки переходов сохраняют эти параметры и изменяют только страницу. Ошибочный запрос получает понятный HTML; пустая выдача остаётся успешным результатом, если условия допустимы.

Jinja2 использует общий базовый шаблон с навигацией и отдельные шаблоны списка, карточки, форм и подтверждений. Пользовательские значения передаются через контекст, как описано в документации FastAPI [13]. Включённое окружением шаблонов автоматическое экранирование обрабатывает HTML-символы; пользовательский текст не помечается safe. Документация Jinja описывает исключения из экранирования и зависимость поведения от настройки [15]. Тесты с введёнными тегами проверяют вывод текста без исполнения разметки.

Для демонстрации большого списка подготовлены 1200 синтетических товаров в 12 категориях. На каждую приходится 100 записей. Отличаются типы, варианты, цены и остатки; 133 товара имеют нулевое количество. Цена находится в диапазоне 19,00–346,10 рубля. Генератор не описывает прайс-лист действующего поставщика. Записи помечены как демонстрационные и имеют стабильные артикулы.

Загрузчик создаёт SQLite backup перед записью в существующую базу. Используется метод backup, описанный в документации sqlite3 [11], а не простое копирование открытого файла. Затем недостающее добавляется одной транзакцией. Имеющийся артикул пропускается, описание существующей категории сохраняется. Поэтому повторный запуск не отменяет пользовательскую правку цены и не создаёт дубль с тем же артикулом. Воспроизводимость набора позволяет проверять те же условия на другом компьютере.

3.3 ТЕСТИРОВАНИЕ, АНАЛИЗ РЕЗУЛЬТАТОВ И ОГРАНИЧЕНИЯ РЕШЕНИЯ

Проверка выполнена 21.10.2026 в облачной Linux-среде на Python 3.12. Автоматические тесты получают отдельные временные SQLite-файлы. Фикстуры создают движок, схему и сессию, а тестовый клиент подменяет зависимость доступа к базе. Такой способ организации согласуется с механизмом фикстур pytest [17]. Рабочий каталог не используется для записи тестовых примеров. Его демонстрационное наполнение выполнено отдельно загрузчиком после резервного копирования.

Всего пройдено 154 теста: 35 проверок базы, 81 API, 31 HTML и 7 загрузчика. Проверяются операции, уникальность, типы и границы чисел, обязательность связи, поиск русского текста, сочетания фильтров, сортировка, пагинация и отказ без частичного изменения. Для загрузчика дополнительно проверены повторное применение, сохранение собственных записей, откат при искусственном сбое и резервная копия с зафиксированными данными WAL.

Сохранено одно предупреждение Starlette о поддерживаемом устаревающем режиме HTTPX в TestClient. Оно не скрывалось и не повлияло на прохождение проверок. Тесты не считаются доказательством отсутствия всех дефектов. Их назначение — проверить определённые ожидания и сохранить возможность повторить проверку после изменения. В Software Engineering at Google такая роль автоматических тестов связывается с поддержкой изменений программы [8, гл. 11].

Дополнительно выполнены 14 сценариев в Chromium 154.0.8037.57 при размерах экрана 1440 × 1000 и 390 × 844. Через Playwright проверены формы, сообщение о неправильной цене с сохранением ввода, совместный отбор, пагинация, защита категории, отмена и подтверждение удаления. Также проверена форма при отключённом JavaScript. Использован синхронный интерфейс автоматизации, соответствующий документации Playwright для Python [19].

Браузерный прогон выявил расширение мобильной страницы скрытой подписью таблицы. Ошибка воспроизведена на узком экране; позиционирование подписи ограничено прокручиваемым контейнером. Повторная проверка сравнила ширину документа с шириной окна. Этот разбор соответствует систематическому подходу к отладке: воспроизвести отказ, уточнить условие и проверить исправление [9]. Также исправлено выравнивание последнего столбца и добавлены проверки слишком больших идентификаторов адресов.

На этом наборе в Chromium выполнены ещё шесть сценариев большого каталога: количество записей API, переход по страницам, последняя страница, совместные фильтры, счётчики категорий и мобильная ширина. Список из 1200 товаров образует 120 страниц; в каждой категории отображается 100 товаров. Итоги сведены в таблицу 5. integrity_check вернул ok, foreign_key_check — отсутствие ошибок; повторная загрузка добавила ноль записей.

Таблица 5 — Фактические результаты проверки

Набор	**Количество**	**Результат**
База данных	35 тестов	Пройдены
JSON API	81 тест	Пройдены
HTML-страницы	31 тест	Пройдены
Загрузчик примеров	7 тестов	Пройдены
Браузер, основные сценарии	14 сценариев	Пройдены в Chromium
Браузер, 1200 товаров	6 сценариев	Пройдены в Chromium
SQLite большого набора	integrity_check, foreign_key_check	ok; ошибок связей нет

Проверка сохранности включает повторное открытие файловой базы другим процессом и отдельный HTTP-прогон с перезапуском на этапе разработки API. В приложении А показаны общий список (рисунок А.1), форма редактирования (рисунок А.2), ошибка цены (рисунок А.3) и запрет удаления

занятой категории (рисунок А.4). Снимки фиксируют реальные страницы: часть относится к раннему сценарию изменения одного товара, часть — к большому набору. Они не выдаются за наблюдение работы действующей организации. Подход к документированию условий и происхождения результата рассматривается в работе Ирвинга и соавторов [4].

Ограничения результатов определяются объёмом проверки. Нагрузочное тестирование, одновременное редактирование несколькими менеджерами и перенос на PostgreSQL не проводились. Отсутствуют авторизация, история движения остатков и миграции схемы. Поэтому полученные результаты подтверждают согласованные локальные сценарии, но не промышленную эксплуатацию. Дальнейшее развитие связано с ролями пользователей, журналом операций, управляемыми миграциями и отдельной проверкой конкурирующих изменений.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута в пределах согласованного учебного модуля: разработано приложение для управления товарами и категориями, их поиска, отбора, сортировки и хранения. Исходная задача разложена на проверяемые пользовательские действия и ограничения. В качестве предметной области использован демонстрационный ассортимент канцелярских товаров. Фактической основой служат модель данных, исходный код, демонстрационный набор и выполненные проверки.

На этапе анализа определены различия между названием, артикулом и техническим идентификатором, смысл цены и текущего остатка, правила связи с категорией. Рассмотрены табличный перечень, готовая система и самостоятельный модуль. Последний выбран в соответствии с задачей изучения Python и FastAPI. Выбор SQLite обоснован локальной демонстрацией и файловой организацией тестов; PostgreSQL рассмотрена по указанной в задании документации без заявления о выполненном переносе.

Спроектированы две основные сущности и связь один ко многим. Категория хранится отдельно от товаров, а её идентификатор используется как внешний ключ. Цена представлена целыми копейками, остаток — неотрицательным целым числом. Нормализованное название используется для поиска без учёта регистра и проверки имени категории. Разделены входные схемы, хранимые модели, общие операции и представления. Это позволило использовать одни правила в HTML и JSON API.

Реализованы десять операций API, карточки и формы, подтверждение удаления, обработка ошибок с сохранением ввода. Поиск сочетается с категорией, включительными ценовыми границами и наличием. Фиксированный перечень сортировок и дополнительный порядок по идентификатору делают выборку однозначной. Пагинация ограничивает страницу десятью записями и сохраняет условия при переходах. Основные

формы работают без JavaScript, а локальное отображение дат дополняется скриптом.

Практический результат дополнен воспроизводимым набором из 1200 товаров. Загрузчик создаёт резервную копию существующей базы, пропускает занятые артикулы и выполняет добавление одной транзакцией. Эти свойства важны для повторной демонстрации: загрузка сохраняет пользовательские изменения. При проверке повторное применение не создало дублей, а проверки целостности SQLite не выявили нарушений.

Функциональная проверка дала 154 успешных автоматических теста и два браузерных набора: 14 основных и 6 дополнительных сценариев большого каталога. Обнаруженные дефекты мобильной ширины и выравнивания таблицы исправлены. Условия, примеры и отчёты сохранены вместе с проектом. Это позволяет сопоставить утверждения записки с исходным кодом и повторяемыми действиями. Вместе с тем количество тестов не означает полного доказательства корректности всех возможных состояний.

Задачи анализа, проектирования, реализации и функционального тестирования выполнены. Разработанный модуль пригоден для демонстрации согласованных локальных сценариев и дальнейшего учебного развития. Для расширения области применения потребуются авторизация, история изменений, миграции и проверка одновременной работы. Пояснительная записка, исходники, демонстрационные данные и материалы защиты образуют комплект результатов, который должен быть проверен студентом и представлен руководителю по установленной процедуре.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201–78. Единая система программной документации. Техническое задание.

2. Московский финансово-юридический университет МФЮА. Методические указания по подготовке и оформлению курсовых работ/проектов. — Москва, 2025. — Текст : электронный.

3. Downey, A. B. Think Python: How to Think Like a Computer Scientist / A. B. Downey. — 3rd ed. — O’Reilly Media, 2024. — ISBN 978-1-098-15543-8. — URL: https://github.com/AllenDowney/ThinkPython/tree/v3 (дата обращения: 21.10.2026). — Текст : электронный.

4. Research Software Engineering with Python: Building Software that Makes Research Possible / D. Irving, K. Hertweck, L. Johnston [et al.]. — Chapman & Hall/CRC Press, 2021. — ISBN 978-0-367-69834-8. — URL: https://github.com/merely-useful/py-rse (дата обращения: 21.10.2026). — Текст : электронный.

5. Percival, H. Architecture Patterns with Python / H. Percival, B. Gregory. — O’Reilly Media, 2020. — ISBN 978-1-492-05220-3. — URL: https://github.com/cosmicpython/book (дата обращения: 21.10.2026). — Текст : электронный.

6. Percival, H. J. W. Test-Driven Development with Python / H. J. W. Percival. — 3rd ed. — O’Reilly Media, 2025. — ISBN 978-1-098-14871-3. — URL: https://github.com/hjwp/Book-TDD-Web-Dev-Python (дата обращения: 21.10.2026). — Текст : электронный.

7. Wilson, G. Software Design by Example: A Tool-Based Introduction with Python / G. Wilson. — URL: https://github.com/gvwilson/sdxpy (дата обращения: 21.10.2026). — Текст : электронный.

8. Winters, T. Software Engineering at Google: Lessons Learned from Programming Over Time / T. Winters, T. Manshreck, H. Wright. — O’Reilly Media, 2020. — ISBN 978-1-492-08279-8. — URL: https://github.com/abseil/abseil.github.io/blob/master/resources/swe-book/html/ch11.html (дата обращения: 21.10.2026). — Текст : электронный.

9. Zeller, A. The Debugging Book / A. Zeller. — 2021. — URL: https://github.com/uds-se/debuggingbook (дата обращения: 21.10.2026). — Текст : электронный.

10. The Fuzzing Book / A. Zeller, R. Gopinath, M. Böhme [et al.]. — 2021. — URL: https://github.com/uds-se/fuzzingbook (дата обращения: 21.10.2026). — Текст : электронный.

11. Python Software Foundation. Python 3.12 Documentation: sqlite3, decimal, unicodedata. — URL: https://github.com/python/cpython/tree/3.12/Doc/library (дата обращения: 21.10.2026). — Текст : электронный.

12. PostgreSQL Global Development Group. PostgreSQL 18 Documentation: Data Definition. — URL: https://github.com/postgres/postgres/blob/REL_18_STABLE/doc/src/sgml/ddl.sgml (дата обращения: 21.10.2026). — Текст : электронный.

13. FastAPI. Templates: официальная документация. — URL: https://github.com/fastapi/fastapi/blob/master/docs/en/docs/advanced/templates.md (дата обращения: 21.10.2026). — Текст : электронный.

14. SQLAlchemy. Session Basics: официальная документация. — URL: https://github.com/sqlalchemy/sqlalchemy/blob/main/doc/build/orm/session_basics.rst (дата обращения: 21.10.2026). — Текст : электронный.

15. Jinja. Template Designer Documentation. — URL: https://github.com/pallets/jinja/blob/main/docs/templates.rst (дата обращения: 21.10.2026). — Текст : электронный.

16. Pydantic. Strict Mode: официальная документация. — URL: https://github.com/pydantic/pydantic/blob/main/docs/concepts/strict_mode.md (дата обращения: 21.10.2026). — Текст : электронный.

17. pytest. How to Use Fixtures: официальная документация. — URL: https://github.com/pytest-dev/pytest/blob/main/doc/en/how-to/fixtures.rst (дата обращения: 21.10.2026). — Текст : электронный.

18. Fielding, R. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. — 2022. — RFC 9110. — URL: https://github.com/httpwg/http-core/blob/main/rfc9110.xml (дата обращения: 21.10.2026). — Текст : электронный.

19. Microsoft. Playwright for Python: Getting Started — Library. — URL: https://github.com/microsoft/playwright/blob/main/docs/src/library-python.md (дата обращения: 21.10.2026). — Текст : электронный.

20. Uvicorn. Settings: официальная документация. — URL: https://github.com/encode/uvicorn/blob/main/docs/settings.md (дата обращения: 21.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А. ЭКРАНЫ И РЕЗУЛЬТАТЫ ДЕМОНСТРАЦИИ

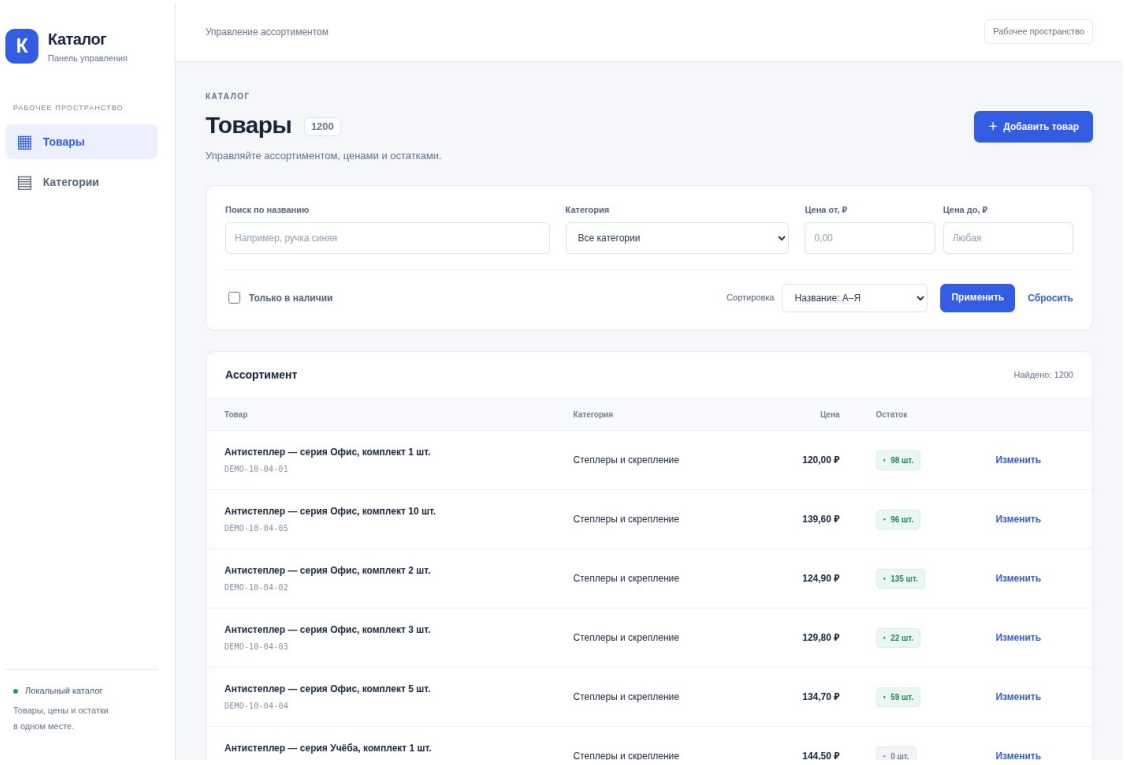


Рисунок А.1 — Каталог из 1200 демонстрационных товаров

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

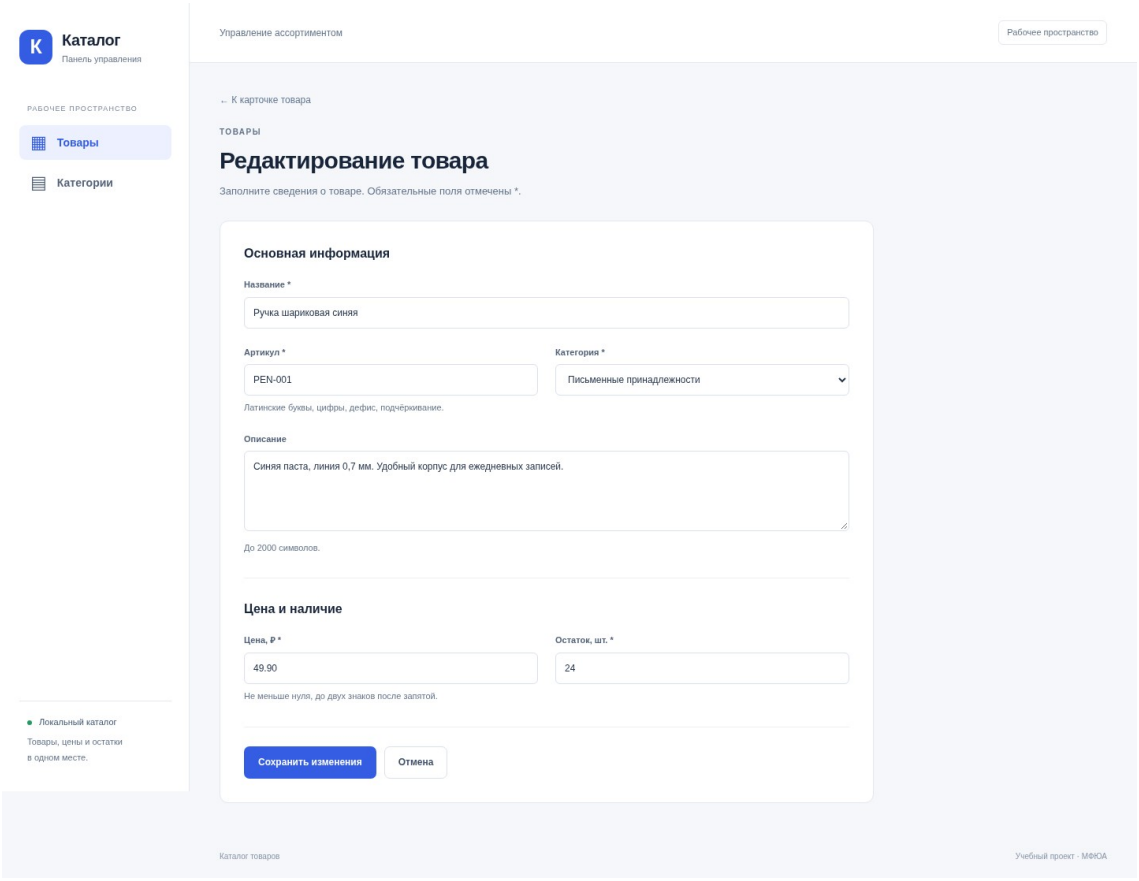


Рисунок А.2 — Форма редактирования товара

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

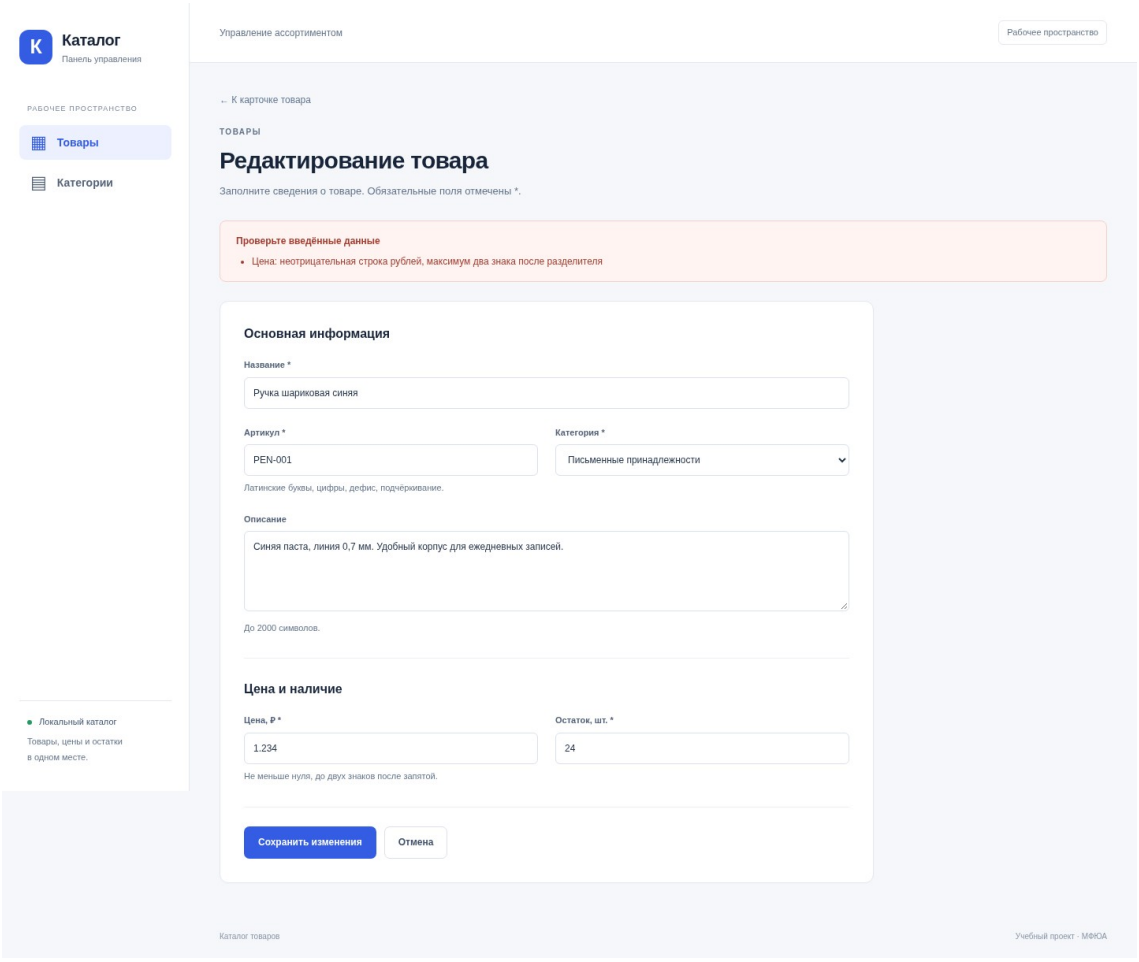


Рисунок А.3 — Сохранение ввода при неправильной цене

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

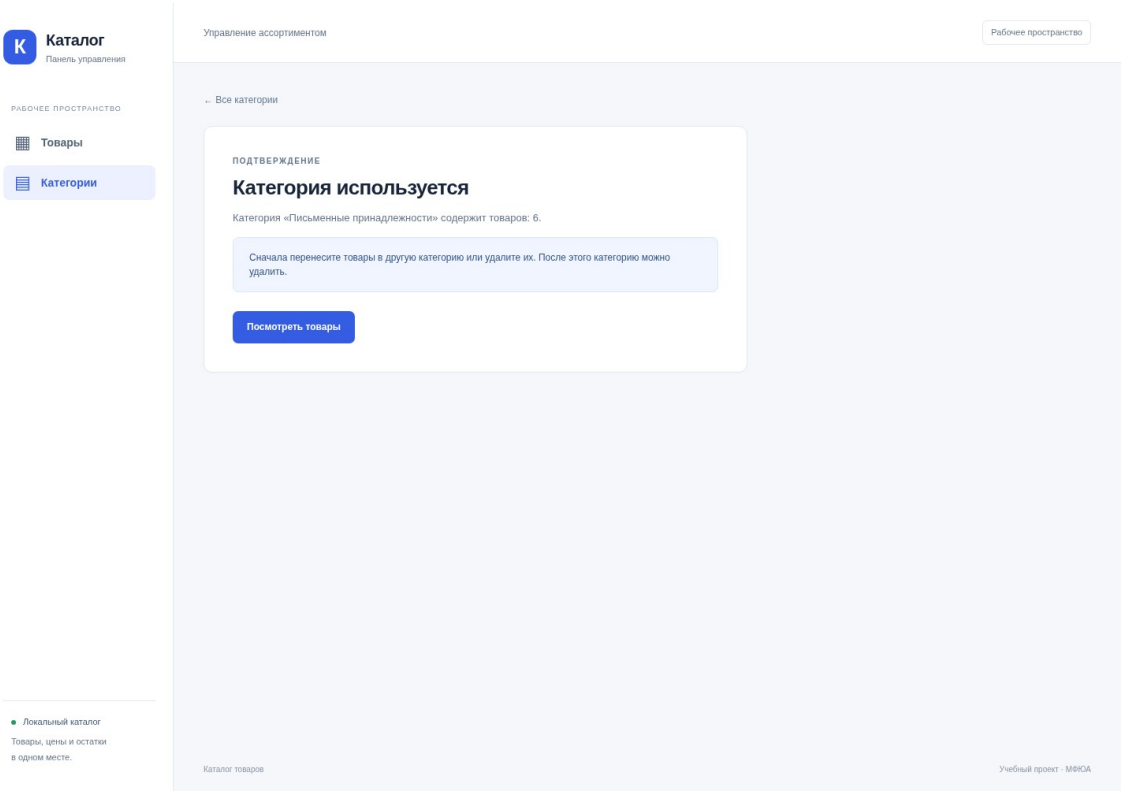


Рисунок А.4 — Запрет удаления категории с товарами

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.